一、BASIC 与 SQL

1. **Relation (关系)**
   A table that conceptually represents a set of tuples (rows) in a database.
2. **Attribute (属性)**
   A column in a table, with a specific domain (set of allowed values) for its data.
3. **Tuple (元组)**
   A row in a table, represented as a list of values corresponding to the table's attributes.
4. **Relation Schema (关系模式)**
   The structure of a relation, defined by its attribute names (e.g., instructor (ID, name, dept_name, salary)).
5. **Relation Instance (关系实例)**
   A specific set of rows in a relation at a given time, representing its current data.
6. **Domain (域)**
   The set of allowed values for an attribute, ensuring data consistency.
7. **Atomicity (原子性)**
   The requirement that attributes are indivisible (not multi-valued or composite).
8. **Null Value (空值)**
   A special value indicating unknown or missing data, considered a member of every attribute domain.

Keys in Relational Models

1. **Superkey (超键)**
   A set of attributes whose values uniquely identify a tuple in any relation instance.
2. **Candidate Key (候选键)**
   A minimal superkey, where no subset of the attributes can uniquely identify a tuple.
3. **Primary Key (主键)**
   A selected candidate key used as the main identifier for tuples in a relation.
4. **Foreign Key (外键)**
   A set of attributes in one relation that references the primary key of another relation.

Relational Algebra: Basic Operations

1. **Select (σ) (选择)**
   Filters tuples from a relation based on a predicate (e.g., σsalary > 85000(instructor)).
2. **Project (Π) (投影)**
   Creates a new relation by selecting specific attributes and removing duplicate rows.
3. **Union (∪) (并)**
   Combines tuples from two compatible relations, removing duplicates.
4. **Set Difference (−) (差集)**
   Returns tuples in the first relation that are not present in the second.
5. **Cartesian Product (×) (笛卡尔积)**
   Combines every tuple from one relation with every tuple from another, resulting in all possible pairs.
6. **Rename (ρ) (重命名)**
   Assigns a new name to a relation or its attributes for easier reference.

Relational Algebra: Additional Operations

1. **Intersection (∩) (交集)**
   Returns tuples that are common to two compatible relations.
2. **Natural Join (⋈) (自然联结)**
   Joins two relations on common attribute names, merging tuples with matching values.
3. **Theta Join (⋈ θ) (θ 联结)**
   Joins relations using a specified predicate, combining a select and cartesian product.
4. **Division (÷) (除)**
   Finds tuples in one relation that match all tuples in another relation based on shared attributes.
5. **Assignment (←) (赋值)**
   Stores the result of a relational expression in a temporary variable to simplify complex queries.

Relational Algebra: Extended Operations

1. **Generalized Projection (泛化投影)**
   Allows arithmetic expressions (e.g., balance * 1.05) in the projection list to derive new values.
2. **Aggregate Functions (聚合函数)**
   Operations (avg, min, max, sum, count) that compute a single value from a group of tuples.
3. **Outer Join (外联结)**
   Preserves unmatched tuples from one or both relations in a join (left, right, full outer join).

Database Modification Operations

1. **Insertion (插入)**
   Adds new tuples to a relation (e.g., $r \leftarrow r \cup E$).
2. **Deletion (删除)**
   Removes tuples from a relation based on a query (e.g., $r \leftarrow r - E$).
3. **Updating (更新)**
   Modifies attribute values in a relation using arithmetic expressions or conditions.

Other Concepts

1. **Schema Diagram (模式图)**
   A visual representation of database relations, attributes, and their connections.
2. **Three-valued Logic (三值逻辑)**
   A logic system using true, false, and unknown to handle null values in predicates.

1. **SQL (结构化查询语言)**
   A standard language for managing relational databases, developed from IBM's Sequel and standardized by ANSI/ISO (e.g., SQL-92, SQL:1999).
1. **Data Definition Language (DDL, 数据定义语言)**
   Used to define database structure, including creating/dropping tables, setting domains, and integrity constraints.
1. **Create Table (创建表)**
   Defines a relation with attributes, data types, and constraints (e.g., CREATE TABLE r (A1 D1, ...)).
1. **Drop Table (删除表)**
   Removes a relation and all its data from the database (e.g., DROP TABLE r).
1. **Alter Table (修改表)**
   Modifies an existing table (e.g., adding/removing attributes or constraints).

Data Types (数据类型)

1. **char(n)**
   Fixed-length character string with length n.
1. **varchar(n)**
   Variable-length character string with max length n.
1. **numeric(p,d)**
   Fixed-point number with p total digits and d decimal digits.
1. **int/smallint**
   Integer types with machine-dependent value ranges.
1. **real/float(n)**
   Floating-point numbers with machine-dependent or user-specified precision.
1. **not null (非空)**
   Prohibits an attribute from having null values.
1. **primary key (主键)**

A unique identifier for tuples, automatically implying not null (e.g., PRIMARY KEY (ID)).

1. **foreign key (外键)**
   References the primary key of another relation to maintain referential integrity (e.g., FOREIGN KEY (dept_name) REFERENCES department).

Query Structure (DML)

1. **Select-From-Where (查询结构)**
   The basic SQL query format: SELECT attrs FROM tables WHERE condition, equivalent to relational algebra projection and selection.
1. **Select Clause (选择子句)**
   Specifies attributes for the result, with DISTINCT to remove duplicates or * for all attributes.
1. **Where Clause (条件子句)**
   Filters tuples using predicates (e.g., WHERE dept_name = 'Comp. Sci.'), supporting AND/OR/NOT and operators like BETWEEN.
1. **From Clause (来源子句)**
   Lists relations involved, implicitly forming a Cartesian product (e.g., FROM instructor, teaches).

Query Operations

1. **Set Operations (集合操作)**
   1. UNION: Combines tuples (duplicates removed by default).
   1. INTERSECT: Returns common tuples.
   1. EXCEPT: Returns tuples in the first set not in the second.
1. **Aggregate Functions (聚合函数)**
   Compute single values from a set of values: AVG, MIN, MAX, SUM, COUNT.
1. **Group By (分组)**
   Groups tuples by attributes to apply aggregate functions (e.g., GROUP BY dept_name).
1. **Having Clause (分组过滤)**
   Filters groups after aggregation (e.g., HAVING AVG(salary) > 42000).

Advanced Query Features

1. **Subquery (子查询)**
   A nested query within another, used in WHERE, FROM, or SELECT clauses.
1. **Exists/Not Exists (存在判断)**
   Tests if a subquery result is non-empty (e.g., EXISTS (SELECT * FROM r)).
1. **Set Comparison (some/all)**
   1. > SOME: True if greater than at least one value.
   1. > ALL: True if greater than all values.
1. **Like (字符串匹配)**
   Uses % (any substring) or _ (any character) for pattern matching (e.g., WHERE name LIKE '%dar%').
1. **Order By (排序)**

Sorts results in ascending (ASC) or descending (DESC) order.

Database Modification (DML)

1. **Insert (插入)**
   Adds tuples to a relation (e.g., INSERT INTO r VALUES (...) or from a query).
1. **Delete (删除)**
   Removes tuples based on a condition (e.g., DELETE FROM r WHERE condition).
1. **Update (更新)**
   Modifies attribute values (e.g., UPDATE r SET attr = expr WHERE condition).

Advanced Features

1. **With Clause (公共表表达式)**
   Defines temporary views for complex queries (e.g., WITH temp AS (SELECT ...) SELECT ...).
1. **View (视图)**
   A virtual relation defined by a query, updating dynamically with base data.
1. **Recursive Query (递归查询)**
   Solves problems involving hierarchical data by referencing itself.

Other Concepts

1. **Cartesian Product (笛卡尔积)**
   Generates all possible tuple pairs from two relations (e.g., FROM r, s).
1. **Rename (重命名)**
   Assigns new names to relations/attributes using AS (e.g., instructor AS T).
1. **Case Statement (条件表达式)**
   Enables conditional updates/queries (e.g., CASE WHEN condition THEN expr END).
1. **API（应用程序接口）**
   A set of tools for programs to connect with a database server, send SQL commands, and fetch results.
1. **ODBC（开放数据库连接）**
   An API for C/C++/C#/Visual Basic to interact with databases, with ADO.NET built on top.
1. **JDBC（Java 数据库连接）**
   A Java API for database connectivity, enabling Java programs to execute SQL queries.
1. **Prepared Statement（预处理语句）**
   A precompiled SQL statement that prevents SQL injection by separating query logic from user inputs.
1. **SQL Injection（SQL 注入）**
   A security vulnerability where malicious inputs manipulate SQL queries, e.g., inserting unauthorized commands.
1. **Transaction Control（事务控制）**
   Managing database transactions to ensure atomicity, using commit() or rollback() in JDBC.
1. **Embedded SQL（嵌入式 SQL）**
   SQL queries embedded within a host language (e.g., C, Java), using constructs like EXEC SQL.
1. **Cursor（游标）**

A database object that allows sequential access to query results in programming languages.

函数与存储过程（Functions and Procedures）

1. **SQL Function（SQL 函数）**
   A user-defined function that returns a value, e.g., counting instructors in a department.
1. **SQL Procedure（SQL 存储过程）**
   A named block of SQL statements that can take input/output parameters, invoked via CALL.
1. **Compound Statement（复合语句）**
   A block of multiple SQL statements enclosed in BEGIN...END.
1. **Table-valued Function（表值函数）**
   A function that returns a relation as its result.
1. **Imperative Constructs（命令式构造）**
   Programming features like loops and IF-THEN-ELSE in SQL.

触发器（Triggers）

1. **Trigger（触发器）**
   A statement automatically executed when a database modification (insert/update/delete) occurs.
1. **Statement-Level Trigger（语句级触发器）**
   A trigger that runs once per SQL statement, using transition tables for all affected rows.
1. **Instead of Trigger（替代触发器）**
   A trigger that replaces the default action for views, enabling updates on complex views.
1. **Transition Tables（过渡表）**
   Temporary tables (OLD TABLE/NEW TABLE) holding rows affected by a statement-level trigger.
1. **External World Actions（外部动作）**
   Non-database actions (e.g., reordering items) triggered by database changes, implemented via a separate process.

其他概念（Other Concepts）

1. **Host Language（宿主语言）**
   A programming language (e.g., Java, C) that embeds SQL queries.
1. **Materialized View（物化视图）**
   A precomputed and stored view used to maintain summary data, replacing trigger-based updates.
1. **Database Replication（数据库复制）**
   Copying data across databases, now often handled by built-in database features instead of triggers.

二、ER 图数据库设计

1. **Initial Phase（初始阶段）**
   The first stage of database design where data needs are characterized by interacting with users and domain experts.
1. **Second Phase（第二阶段）**
   Choosing a data model and translating requirements into a conceptual schema of the database.
1. **Final Phase（最终阶段）**
   Moving from an abstract data model to the implementation of the database, including logical and physical design.
1. **Logical Design（逻辑设计）**
   Deciding on the database schema, including relation schemas and attribute distribution.
1. **Physical Design（物理设计）**
   Determining the physical layout of the database, such as storage structures and indices.

实体 - 关系模型基础（ER Model Basics）

1. **Entity（实体）**
   An object that exists and is distinguishable from other objects, such as a person or event.
1. **Entity Set（实体集）**
   A set of entities of the same type that share common properties, e.g., all instructors or students.
1. **Attribute（属性）**
   A descriptive property of an entity, such as name or salary, with a specific domain of values.
1. **Relationship（关系）**
   An association among entities, representing how they are related to each other.
1. **Relationship Set（关系集）**
   A set of relationships among entities from one or more entity sets, e.g., the advisor relationship between instructors and students.

属性类型（Attribute Types）

1. **Simple Attribute（简单属性）**
   An indivisible attribute, such as an ID or age.
1. **Composite Attribute（复合属性）**
   An attribute composed of subparts, like an address structured as street, city, and state.
1. **Single-valued Attribute（单值属性）**
   An attribute that has exactly one value for each entity, e.g., a person’s date of birth.
1. **Multi-valued Attribute（多值属性）**
   An attribute that can have multiple values for an entity, such as a person’s phone numbers.
1. **Derived Attribute（派生属性）**
   An attribute that can be computed from other attributes, like age derived from date of birth.
1. **Stored Attribute（存储属性）**
   An attribute whose value is stored in the database, as opposed to being derived.

关系集特性（Relationship Set Features）

1. **Degree of Relationship Set（关系集的度）**
   The number of entity sets participating in a relationship set, with binary (two sets) being the most common.
1. **Mapping Cardinality（映射基数）**
   Describes the number of entities in one set that can be associated with entities in another set, including one-to-one, one-to-many, many-to-one, and many-to-many.
1. **Participation Constraint（参与约束）**
   Specifies whether an entity must participate (total participation) or may not participate (partial participation) in a relationship set.
1. **Total Participation（完全参与）**
   Every entity in the set must participate in at least one relationship in the set.
1. **Partial Participation（部分参与）**
   Some entities may not participate in any relationship in the set.
1. **Role（角色）**
   A label clarifying an entity's role in a relationship, used when entity sets in a relationship are the same (e.g., course and prereq course).

键与约束（Keys and Constraints）

1. **Super Key（超键）**
   A set of attributes whose values uniquely identify an entity in an entity set.
1. **Candidate Key（候选键）**
   A minimal super key, where no subset can uniquely identify an entity.
1. **Primary Key（主键）**
   A selected candidate key used to uniquely identify entities, chosen from available candidates.
1. **Keys for Relationship Sets（关系集的键）**
   Super keys for a relationship set, often formed by combining primary keys of participating entity sets.

ER 图表示（ER Diagram Notation）

1. **ER Diagram（ER 图）**
   A visual representation of the ER model, using rectangles for entity sets, diamonds for relationship sets, and ovals for attributes.
1. **Cardinality Constraints in ERD（ER 图中的基数约束）**
   Represented by directed lines (→) for "one" and undirected lines (—) for "many" between entity and relationship sets.
1. **Participation Notation（参与约束表示）**
   Total participation shown with double lines, partial participation with single lines.
1. **Composite Attribute in ERD（ER 图中的复合属性）**
   Represented by nesting attributes, such as address broken into street, city, etc.
1. **Multi-valued Attribute in ERD（ER 图中的多值属性）**
   Denoted by double ellipses, e.g., {phone_number}.

1. **Derived Attribute in ERD（ER 图中的派生属性）**
   Represented by dashed ellipses, e.g., age() derived from date_of_birth.

设计问题与高级概念（Design Issues and Advanced Concepts）

1. **Entity Set vs. Attribute（实体集与属性的选择）**
   Deciding whether to model a concept as an entity set (e.g., phone) or an attribute (e.g., phone_number) based on semantics.
1. **Entity Set vs. Relationship Set（实体集与关系集的选择）**
   Choosing between modeling an association as a relationship set (e.g., takes) or an entity set (e.g., registration).
1. **Binary vs. n-ary Relationship Set（二元与 n 元关系集）**
   Deciding between using a binary relationship (two entity sets) or a non-binary (n-ary) relationship (three or more sets), with n-ary sets being less common.
1. **Ternary Relationship Set（三元关系集）**
   A relationship set involving three entity sets, such as proj_guide between instructor, student, and project.
1. **Converting n-ary to Binary Relationships（n 元关系转换为二元关系）**
   Replacing an n-ary relationship with binary relationships using an artificial entity set to preserve constraints.
1. **Placement of Relationship Attributes（关系属性的放置）**
   Deciding whether an attribute belongs to a relationship set (e.g., date in advisor) or an entity set, based on mapping cardinality.

常见错误（Common Mistakes）

1. **Using Primary Key as Attribute（将主键作为属性使用）**
   Incorrectly including the primary key of one entity set as an attribute of another, instead of using a relationship.
1. **Incorrect Attribute Type（错误的属性类型）**
   Using a single-valued attribute where a multi-valued attribute is needed, such as marks for multiple assignments.

弱实体集（Weak Entity Sets）

1. **Weak Entity Set（弱实体集）**
   An entity set that lacks a primary key and depends on an identifying strong entity set for its existence.
1. **Identifying Relationship Set（标识关系集）**
   A total, one-to-many relationship that links a weak entity set to its identifying strong entity set.
1. **Discriminator（ discriminator，区分符）**
   A partial key of a weak entity set that, combined with the primary key of the identifying entity set, forms a primary key.
1. **Weak Entity Set Notation（弱实体集表示法）**

Represented by a double rectangle in ER diagrams, with discriminators underlined by a dashed line.

1. **Specialization（特化）**
   A top-down design process defining subgroupings within an entity set (e.g., student and employee as subclasses of person).
1. **Generalization（泛化）**
   A bottom-up process combining entity sets into a higher-level superclass (e.g., person as a superclass of student and employee).
1. **Superclass and Subclass（超类与子类）**
   Higher-level (superclass) and lower-level (subclass) entity sets in a specialization/generalization hierarchy.
1. **Attribute Inheritance（属性继承）**
   Subclasses inherit attributes and relationship participations from their superclass.
1. **Condition-defined Specialization（条件定义特化）**
   Subclasses defined by explicit conditions (e.g., senior-citizen as persons over 65).
1. **User-defined Specialization（用户定义特化）**
   Subclasses assigned by user decision (e.g., work teams after a novitiate).
1. **Disjoint Constraint（不相交约束）**
   Entities can belong to only one subclass within a generalization.
1. **Overlapping Constraint（重叠约束）**
   Entities can belong to multiple subclasses within a generalization.
1. **Total Generalization（完全泛化）**
   All entities in the superclass must belong to at least one subclass.
1. **Partial Generalization（部分泛化）**
   Entities may not belong to any subclass within the superclass.
1. **Aggregation（聚合）**
   Treating a relationship set as an abstract entity to form higher-level relationships (e.g., evaluating a student-instructor-project relationship).

ER 到关系模式的转换（Reduction to Relation Schemas）

1. **Strong Entity to Schema（强实体集到模式）**
   A strong entity set converts to a schema with its attributes (e.g., course(course_id, title, credits)).
1. **Weak Entity to Schema（弱实体集到模式）**
   A weak entity set schema includes the primary key of its identifying entity (e.g., section(course_id, sec_id, semester, year)).
1. **Composite Attribute Flattening（复合属性展开）**
   Composite attributes convert to separate schema attributes (e.g., name → first_name, last_name).
1. **Multivalued Attribute to Schema（多值属性到模式）**
   A multivalued attribute becomes a separate schema with the entity’s primary key (e.g., inst_phone(ID, phone_number)).
1. **Relationship Set to Schema（关系集到模式）**
   A many-to-many relationship converts to a schema with primary keys of participating entities (e.g., advisor(s_id, i_id)).
1. **Redundant Schema Elimination（冗余模式消除）**

Many-to-one relationships can be represented by adding a foreign key to the “many” side instead of a separate schema.

1. **Specialization to Schemas（特化到模式）**
   Subclasses can be represented by separate schemas with inherited attributes or by merging attributes into subclass schemas.

UML 与 ER 图（UML and ER Diagrams）

1. **UML Class Diagram（UML 类图）**
   A graphical model similar to ER diagrams but with differences in notation for entities, relationships, and constraints.
1. **Cardinality in UML（UML 中的基数）**
   Specified as min..max (e.g., 0..* for zero or many), reversed from ER diagram notation.
1. **Generalization in UML（UML 中的泛化）**
   Represented by arrows from subclasses to superclasses, with disjoint/overlapping and total/partial constraints.

其他概念（Other Concepts）

1. **Design Decisions（设计决策）**
   Choices in modeling, such as using attributes vs. entity sets, binary vs. n-ary relationships, or weak vs. strong entities.
1. **ER Diagram Symbols（ER 图符号）**
   Notations for entity sets, relationship sets, attributes, keys, and constraints in ER diagrams.

三、关系数据库设计基础

1. **Good Relational Design（良好关系设计）**
   A design that minimizes data redundancy and ensures information can be retrieved efficiently without loss.
1. **Bad Relational Design（糟糕关系设计）**
   A design that causes data repetition or inability to represent information, often due to improper schema decomposition.
1. **Schema Decomposition（模式分解）**
   The process of splitting a relation schema into smaller schemas to eliminate redundancy, requiring lossless and dependency-preserving properties.
1. **Lossless Decomposition（无损分解）**
   A decomposition where the original relation can be reconstructed by joining the decomposed relations, ensuring no information loss.
1. **Dependency Preservation（依赖保持）**
   A decomposition that allows all original functional dependencies to be enforced by checking individual decomposed relations.

函数依赖（Functional Dependencies）

1. **Functional Dependency (FD, 函数依赖)**
   A constraint where the value of one attribute set uniquely determines another (e.g., $\alpha \rightarrow \beta$ means α determines β).
1. **Trivial FD（平凡函数依赖）**
   A dependency where $\beta \subseteq \alpha$ (e.g., ID, name → ID), satisfied by all relation instances.
1. **Nontrivial FD（非平凡函数依赖）**
   A dependency where $\beta \not\subseteq \alpha$, requiring explicit constraint enforcement.
1. **Closure of FDs (F+, 函数依赖闭包)**
   The set of all functional dependencies logically implied by a given set F, derivable using Armstrong’s axioms.
1. **Armstrong’s Axioms（阿姆斯特朗公理）**
   A set of rules (reflexivity, augmentation, transitivity) for deriving all implied functional dependencies.
1. **Attribute Closure（属性集闭包）**
   The set of attributes functionally determined by a given attribute set α, denoted $\alpha+$, used to test superkeys and FD validity.
1. **Canonical Cover（正则覆盖）**
   A minimal set of functional dependencies equivalent to F, with no extraneous attributes or redundant dependencies.

范式（Normal Forms）

1. **First Normal Form (1NF, 第一范式)**

A relation where all attribute domains are atomic (indivisible), e.g., no multi-valued or composite attributes.

1. **Boyce-Codd Normal Form (BCNF, 巴斯 - 科德范式)**
   A relation where every non-trivial FD has a superkey on the left side, eliminating most redundancy.
1. **Third Normal Form (3NF, 第三范式)**
   A weaker form of BCNF allowing non-trivial FDs where non-key attributes depend on a superkey or are part of a candidate key.

范式应用与分解（Normal Form Application and Decomposition）

1. **BCNF Decomposition Algorithm（BCNF 分解算法）**
   A process to decompose a schema into BCNF by splitting on non-trivial FDs where the left side is not a superkey.
1. **3NF Decomposition Algorithm（3NF 分解算法）**
   A method to decompose a schema into 3NF while preserving dependencies and ensuring lossless join.
1. **Superkey（超键）**
   An attribute set whose values uniquely identify a tuple (e.g., K → R for schema R).
1. **Candidate Key（候选键）**
   A minimal superkey, with no subset being a superkey.
1. **Primary Key（主键）**
   A selected candidate key used as the main identifier for a relation.

分解特性（Decomposition Properties）

1. **Lossy Decomposition（有损分解）**
   A decomposition where joining decomposed relations yields more tuples than the original, causing information loss.
1. **Dependency Preservation Test（依赖保持测试）**
   A check to ensure all original FDs can be enforced on decomposed relations without joining.

其他概念（Other Concepts）

1. **Extraneous Attribute（无关属性）**
   An attribute in an FD that can be removed without changing the closure of the dependency set.
1. **Multivalued Dependency (MVD, 多值依赖)**
   A constraint where one attribute set determines another independent of a third, addressing remaining redundancy in BCNF relations.

数据库设计目标与流程（Design Goals and Process）

1. **Design Goals（设计目标）**
   Achieving BCNF with lossless and dependency-preserving decomposition, prioritizing minimal redundancy and efficient querying.
1. **Overall Design Process（整体设计流程）**
   Converting E-R diagrams to relations, normalizing schemas, and addressing functional dependencies to avoid anomalies.
1. **Denormalization（反规范化）**
   Deliberately introducing redundancy for performance optimization, such as combining tables to speed up queries.
1. **Materialized View（物化视图）**
   A precomputed and stored view that improves query performance but requires maintenance on updates.

范式进阶（Advanced Normal Forms）

1. **Second Normal Form (2NF, 第二范式)**
   A relation in 1NF where nonprime attributes are fully dependent on every candidate key, eliminating partial dependencies.
1. **Partial Dependency（部分依赖）**
   A nonprime attribute depending on only a subset of a composite key, violating 2NF.
1. **Third Normal Form (3NF, 第三范式)**
   A relation in 2NF with no transitive dependencies of nonprime attributes on candidate keys.
1. **Transitive Dependency（传递依赖）**
   A nonprime attribute depending on another nonprime attribute, which in turn depends on a key, violating 3NF.
1. **Boyce-Codd Normal Form (BCNF, 巴斯 - 科德范式)**
   A relation where every non-trivial functional dependency has a superkey on the left side, eliminating remaining redundancy from 3NF.
1. **Fourth Normal Form (4NF, 第四范式)**
   A relation free of non-trivial multivalued dependencies, requiring every multivalued dependency to be a superkey.

多值依赖与分解测试（Multivalued Dependencies and Decomposition Tests）

1. **Multivalued Dependency (MVD, 多值依赖)**
   A constraint where one attribute set determines another independently of a third, e.g., $\alpha \twoheadrightarrow \beta$ meaning β is independent of $R-\alpha-\beta$.
1. **Chase Procedure（追逐算法）**
   A test to determine if a decomposition is lossless by checking if a row of all primary attributes can be formed through FD enforcement.
1. **Lossless Decomposition Test（无损分解测试）**
   Using the chase procedure to verify if joining decomposed relations reconstructs the original schema.

其他概念（Other Concepts）

1. **Prime Attribute（主属性）**
   An attribute that is part of any candidate key of a relation.
1. **Nonprime Attribute（非主属性）**
   An attribute that is not part of any candidate key.
1. **Domain-Key Normal Form (DKNF, 域键范式)**
   The highest normal form, ensuring all constraints are implied by domain definitions and key constraints, though rarely used due to complexity.
1. **Project-Join Normal Form (PJNF, 投影连接范式)**
   A form ensuring every join dependency is a consequence of candidate keys, also known as fifth normal form (5NF).

四、事务基础（Transaction Basics）

1. **Transaction（事务）**
   A unit of program execution that accesses and updates data, ensuring the database remains consistent before and after its execution.
1. **ACID Properties（ACID 特性）**
   1. **Atomicity（原子性）**：Either all transaction operations complete or none, preventing partial updates.
   1. **Consistency（一致性）**：Transactions maintain database consistency when executed in isolation.
   1. **Isolation（隔离性）**：Concurrent transactions appear to execute serially, hiding intermediate states.
   1. **Durability（持久性）**：Committed changes persist even after system failures.
1. **Transaction State（事务状态）**
   1. **Active（活动）**：Executing transactions.
   1. **Partially Committed（部分提交）**：Final statement executed, waiting for commit.
   1. **Failed（失败）**：Execution cannot proceed normally.
   1. **Aborted（中止）**：Rolled back to initial state.
   1. **Committed（提交）**：Successfully completed.

并发执行与调度（Concurrent Execution and Schedules）

1. **Schedule（调度）**
   A sequence of operations from concurrent transactions, preserving each transaction's internal order.
1. **Serial Schedule（串行调度）**
   Transactions execute one after another, ensuring consistency trivially.
1. **Concurrent Schedule（并发调度）**
   Transactions execute interleaved, requiring controls to maintain consistency.
1. **Precedence Graph（优先图）**
   A directed graph showing transaction dependencies to test conflict serializability.

可串行化（Serializability）

1. **Serializability（可串行化）**
   A concurrent schedule is equivalent to a serial schedule, maintaining database consistency.
1. **Conflict Serializability（冲突可串行化）**
   A schedule transformable to a serial schedule via non-conflicting operation swaps.
1. **Conflicting Operations（冲突操作）**
   Two operations on the same data item where at least one is a write, forcing an order.
1. **View Serializability（视图可串行化）**
   A weaker form of serializability based on read/write dependencies, allowing blind writes.

恢复与并发控制（Recovery and Concurrency Control）

1. **Recoverable Schedule（可恢复调度）**

A schedule where a transaction reading data from another transaction commits after the latter.

1. **Cascading Rollback（级联回滚）**
   A transaction failure causing subsequent transactions to roll back, due to dependencies.
1. **Cascadeless Schedule（无级联调度）**
   Prevents cascading rollbacks by ensuring transactions read committed data.
1. **Concurrency Control（并发控制）**
   Mechanisms to ensure serializable and recoverable schedules, like locking or timestamping.

SQL 中的事务（Transactions in SQL）

1. **Transaction Definition in SQL**
   Transactions begin implicitly, ending with COMMIT WORK (commit) or ROLLBACK WORK (abort).
1. **Isolation Levels（隔离级别）**
   1. **Serializable（可串行化）：** Default, ensures serializable schedules.
   1. **Repeatable Read（可重复读）：** Repeated reads return the same value, but may miss new data.
   1. **Read Committed（读已提交）：** Reads only committed data, but values may change.
   1. **Read Uncommitted（读未提交）：** Reads uncommitted data, risking inconsistency.

五、索引基础（Index Basics）

1. **Index（索引）**
   A data structure that speeds up data retrieval by organizing search keys with pointers to records.
1. **Search Key（搜索键）**
   An attribute or set of attributes used to look up records in a file.
1. **Index Entry（索引项）**
   A record in an index file containing a search key and a pointer to the corresponding data record.
1. **Ordered Index（有序索引）**
   An index where search keys are stored in sorted order, enabling efficient range queries.
1. **Hash Index（哈希索引）**
   An index using a hash function to map search keys to bucket addresses for direct record access.

有序索引类型（Types of Ordered Indices）

1. **Clustering Index（聚簇索引）**
   An index where the file is ordered by the index key, also called a primary index.
1. **Nonclustering Index（非聚簇索引）**
   An index where the file order differs from the index key order, also called a secondary index.
1. **Dense Index（稠密索引）**
   An index with an entry for every search key value in the file.
1. **Sparse Index（稀疏索引）**
   An index with entries for only some search key values, applicable to sequentially ordered files.
1. **Multilevel Index（多级索引）**
   A hierarchy of indices where each level indexes the level below, reducing disk accesses.

B + 树索引（B+Tree Index）

1. **B+Tree Index（B + 树索引）**
   A balanced tree structure allowing efficient insertions, deletions, and range queries, with all leaf nodes at the same level.
1. **B+Tree Node（B + 树节点）**
   A node in a B+ tree containing search keys and pointers, with leaf nodes pointing to records and non-leaf nodes pointing to child nodes.
1. **B+Tree Properties（B + 树特性）**
   1. All leaf nodes are at the same level.
   1. Nodes have a minimum and maximum number of children/keys to maintain balance.

哈希索引（Hashing）

1. **Hash Function（哈希函数）**

A function mapping search keys to bucket addresses, ideally distributing keys uniformly.

1. **Bucket（桶）**
   A storage unit (e.g., disk block) containing records mapped to the same hash value.
1. **Static Hashing（静态哈希）**
   A hashing method with a fixed number of buckets, prone to overflow as data grows.
1. **Overflow Chaining（溢出链）**
   A technique using linked lists to handle bucket overflows when multiple records map to the same bucket.
1. **Dynamic Hashing（动态哈希）**
   A hashing method allowing the number of buckets to adjust dynamically, avoiding periodic reorganization.

高级索引技术（Advanced Indexing Techniques）

1. **Multiple-Key Index（多键索引）**
   An index on a combination of attributes, optimizing queries with multiple conditions.
1. **Bitmap Index（位图索引）**
   An index using bit arrays where each bit represents a record’s presence for a specific attribute value, efficient for multi-attribute queries.
1. **Bitmap Operations（位图操作）**
   Bitwise operations (AND, OR, NOT) on bitmap indices to combine query results efficiently.

SQL 中的索引（Indices in SQL）

1. **Index Definition in SQL**
   Creating indices using CREATE INDEX, e.g., CREATE INDEX idx ON table(col) to speed up queries.
1. **Unique Index（唯一索引）**
   An index ensuring no duplicate search key values, used to enforce uniqueness constraints.
1. **Bitmap Index in Oracle**
   A specialized index in Oracle for efficient multi-attribute queries on low-cardinality attributes.

索引设计与优化（Index Design and Optimization）

1. **Index Selection（索引选择）**
   Choosing indices based on query frequency, attribute selectivity, and update frequency.
1. **Index Performance Considerations**
   Balancing query speed improvements against insertion/deletion overhead and space usage.
1. **Index Rebuilding（索引重建）**
   Re
1. **Index（索引）**
   A data structure that speeds up data retrieval by organizing search keys with pointers to records.
1. **Search Key（搜索键）**
   An attribute or set of attributes used to look up records in a file.
1. **Index Entry（索引项）**

A record in an index file containing a search key and a pointer to the corresponding data record.

1. **Ordered Index（有序索引）**
   An index where search keys are stored in sorted order, enabling efficient range queries.
1. **Hash Index（哈希索引）**
   An index using a hash function to map search keys to bucket addresses for direct record access.

有序索引类型（Types of Ordered Indices）

1. **Clustering Index（聚簇索引）**
   An index where the file is ordered by the index key, also called a primary index.
1. **Nonclustering Index（非聚簇索引）**
   An index where the file order differs from the index key order, also called a secondary index.
1. **Dense Index（稠密索引）**
   An index with an entry for every search key value in the file.
1. **Sparse Index（稀疏索引）**
   An index with entries for only some search key values, applicable to sequentially ordered files.
1. **Multilevel Index（多级索引）**
   A hierarchy of indices where each level indexes the level below, reducing disk accesses.

B + 树索引（B+Tree Index）

1. **B+Tree Index（B + 树索引）**
   A balanced tree structure allowing efficient insertions, deletions, and range queries, with all leaf nodes at the same level.
1. **B+Tree Node（B + 树节点）**
   A node in a B+ tree containing search keys and pointers, with leaf nodes pointing to records and non-leaf nodes pointing to child nodes.
1. **B+Tree Properties（B + 树特性）**
   1. All leaf nodes are at the same level.
   1. Nodes have a minimum and maximum number of children/keys to maintain balance.

哈希索引（Hashing）

1. **Hash Function（哈希函数）**
   A function mapping search keys to bucket addresses, ideally distributing keys uniformly.
1. **Bucket（桶）**
   A storage unit (e.g., disk block) containing records mapped to the same hash value.
1. **Static Hashing（静态哈希）**
   A hashing method with a fixed number of buckets, prone to overflow as data grows.
1. **Overflow Chaining（溢出链）**
   A technique using linked lists to handle bucket overflows when multiple records map to the same bucket.
1. **Dynamic Hashing（动态哈希）**

A hashing method allowing the number of buckets to adjust dynamically, avoiding periodic reorganization.

高级索引技术（Advanced Indexing Techniques）

1. **Multiple-Key Index（多键索引）**
   An index on a combination of attributes, optimizing queries with multiple conditions.
1. **Bitmap Index（位图索引）**
   An index using bit arrays where each bit represents a record’s presence for a specific attribute value, efficient for multi-attribute queries.
1. **Bitmap Operations（位图操作）**
   Bitwise operations (AND, OR, NOT) on bitmap indices to combine query results efficiently.

SQL 中的索引（Indices in SQL）

1. **Index Definition in SQL**
   Creating indices using CREATE INDEX, e.g., CREATE INDEX idx ON table(col) to speed up queries.
1. **Unique Index（唯一索引）**
   An index ensuring no duplicate search key values, used to enforce uniqueness constraints.
1. **Bitmap Index in Oracle**
   A specialized index in Oracle for efficient multi-attribute queries on low-cardinality attributes.

索引设计与优化（Index Design and Optimization）

1. **Index Selection（索引选择）**
   Choosing indices based on query frequency, attribute selectivity, and update frequency.
1. **Index Performance Considerations**
   Balancing query speed improvements against insertion/deletion overhead and space usage.
1. **Index Rebuilding（索引重建）**
   Re

六、查询处理基础（Query Processing Basics）

1. **Query Processing（查询处理）**
   The process of parsing, optimizing, and executing database queries, including translation to relational algebra and plan evaluation.
1. **Parsing and Translation（解析与转换）**
   Converting a query into an internal form and then to relational algebra, with syntax checking and relation verification.
1. **Query Optimization（查询优化）**
   Selecting the most efficient execution plan among equivalent relational algebra expressions using cost estimation.
1. **Evaluation Plan（执行计划）**
   A detailed strategy specifying algorithms for each operation in a query, annotated with execution steps.

查询成本度量（Query Cost Measures）

1. **Query Cost（查询成本）**
   Measured by disk I/O (blocks read/written, seeks), CPU usage, and network communication, with disk I/O often prioritized for estimation.
1. **Response Time（响应时间）**
   The total elapsed time to answer a query, harder to estimate but critical for user experience.
1. **Resource Consumption（资源消耗）**
   Total system resources used (e.g., disk and CPU), suitable for shared database environments.

查询操作实现（Query Operation Implementation）

1. **Selection Operation（选择操作）**
   Retrieving tuples that satisfy a condition, implemented via file scans or index-based searches.
   1. **File Scan（文件扫描）：** Sequential scan of all blocks, suitable for unindexed data.
   1. **Index Scan（索引扫描）：** Using indices (e.g., B + 树) to locate tuples efficiently.
1. **Sorting（排序）**
   Ordering tuples, often using external sort-merge for large datasets, which splits data into sorted runs and merges them.
1. **Join Operation（连接操作）**
   Combining tuples from multiple relations, with algorithms like:
   1. **Nested-Loop Join（嵌套循环连接）：** Iterating over all tuple pairs, simple but expensive.
   1. **Block Nested-Loop Join（块嵌套循环连接）：** Processing blocks instead of tuples to reduce I/O.
   1. **Indexed Nested-Loop Join（索引嵌套循环连接）：** Using indices to speed up inner relation lookups.
   1. **Merge-Join（归并连接）：** Requiring sorted inputs, merging them efficiently.
   1. **Hash-Join（哈希连接）：** Partitioning tuples by hash values to avoid full scans.

查询优化技术（Query Optimization Techniques）

1. **Equivalence Rules（等价规则）**
   Rules to transform relational algebra expressions into equivalent forms, e.g., commutativity/associativity of joins and selections.
1. **Pushdown of Selections/Projections（选择 / 投影下推）**
   Moving selection/projection operations early in the query plan to reduce input size for subsequent operations.
1. **Join Ordering（连接顺序）**
   Choosing the optimal order of joining relations to minimize intermediate result size, often using dynamic programming for large queries.
1. **Cost-based Optimization（基于成本的优化）**
   Estimating costs of different plans using statistics (tuple counts, attribute distributions) to select the cheapest option.
1. **Heuristic Optimization（启发式优化）**
   Using rules of thumb (e.g., perform restrictive selections first) to simplify optimization without exhaustive cost analysis.

嵌套子查询优化（Nested Subquery Optimization）

1. **Correlated Subquery（相关子查询）**
   A subquery that references variables from the outer query, often inefficient due to repeated execution.
1. **Decorrelation（去关联）**
   Transforming nested subqueries into joins or semijoins to enable efficient execution using join algorithms.
1. **Semijoin/Anti-semijoin（半连接 / 反半连接）**
   1. **Semijoin（半连接）**：Retaining outer tuples with at least one match in the inner relation.
   1. **Anti-semijoin（反半连接）**：Retaining outer tuples with no matches in the inner relation.

统计信息与优化工具（Statistics and Optimization Tools）

1. **Database Catalog（数据库目录）**
   Storing metadata like tuple counts, attribute statistics, and index information for cost estimation.
1. **Explain Plan（执行计划解释）**
   A tool displaying the query optimizer’s chosen plan and cost estimates, available in databases like Oracle and SQL Server.

七、物理存储介质（Physical Storage Media）

1. **Volatile Storage（易失性存储）**
   Storage that loses data when power is off, such as main memory and cache.
1. **Non-volatile Storage（非易失性存储）**
   Storage that retains data without power, including secondary (disks) and tertiary (tapes) storage.
1. **Storage Hierarchy（存储层次结构）**
   A hierarchy from fast/expensive (cache) to slow/cheap (tapes), balancing speed and cost.
1. **Primary Storage（主存储）**
   Fast but volatile storage (cache, main memory) for active data processing.
1. **Secondary Storage（二级存储）**
   Non-volatile, moderate-speed storage (magnetic disks, SSDs) for long-term data storage.
1. **Tertiary Storage（三级存储）**
   Slow, low-cost storage (tapes, optical disks) for archival and backup.

磁盘与存储技术（Disks and Storage Technologies）

1. **Magnetic Disk（磁盘）**
   A primary non-volatile storage medium using spinning platters and read/write heads for data storage.
1. **SSD (Solid State Disk, 固态硬盘)**
   A non-volatile storage using flash memory, offering faster I/O than magnetic disks.
1. **Optical Storage（光存储）**
   Non-volatile storage using lasers (CD/DVD/BD), suitable for archival with lower cost but slower access.
1. **Tape Storage（磁带存储）**
   Sequential-access storage for backup and archiving, offering high capacity at low cost.
1. **Disk Controller（磁盘控制器）**
   Manages disk operations, including sector read/write, checksum verification, and bad sector remapping.
1. **Storage Interfaces（存储接口）**
   Standards like SATA, SAS, and NVMe defining how disks connect to systems, affecting transfer speeds.
1. **RAID (Redundant Arrays of Independent Disks, 磁盘阵列)**
   Techniques using multiple disks for higher capacity, speed, and reliability via striping and redundancy.
1. **RAID Levels（RAID 级别）**
   Different configurations (e.g., RAID 0, 1, 5) balancing performance, redundancy, and cost.

磁盘性能与优化（Disk Performance and Optimization）

1. **Access Time（访问时间）**
   The time to read/write data, including seek time (arm movement) and rotational latency (platter rotation).
1. **Data-transfer Rate（数据传输率）**

The speed at which data is read from/written to disk, affected by disk location and interface.

1. **IOPS (I/O Operations Per Second, 每秒输入输出操作数)**
   A measure of disk performance for random small-block accesses.
1. **MTTF (Mean Time To Failure, 平均故障间隔时间)**
   The average time a disk runs without failure, indicating reliability.
1. **Disk Arm Scheduling（磁盘臂调度）**
   Algorithms (e.g., elevator algorithm) to minimize disk arm movement for better performance.
1. **Disk Defragmentation（磁盘碎片整理）**
   Rearranging data to contiguous blocks to improve access speed for fragmented files.
1. **Nonvolatile Write Buffer（非易失性写缓冲区）**
   A battery-backed buffer allowing fast writes to disk, improving I/O performance.

文件组织与记录存储（File Organization and Record Storage）

1. **Fixed-Length Records（定长记录）**
   Records of uniform size stored sequentially, simplifying access but wasting space if variable data.
1. **Variable-Length Records（变长记录）**
   Records with varying sizes, using headers to track offsets and lengths for efficient storage.
1. **Slotted Page Structure（页槽结构）**
   A block format with a header tracking record locations, enabling variable-length records in a block.
1. **File Organization（文件组织）**
   Methods to store records: heap (unordered), sequential (sorted), or clustered (related records together).
1. **Heap File Organization（堆文件组织）**
   Records stored anywhere in the file, using a free-space map to manage available space.
1. **Sequential File Organization（顺序文件组织）**
   Records sorted by a key, efficient for sequential access but requiring periodic reorganization.
1. **Multitable Clustering（多表聚簇）**
   Storing related records from multiple tables together to reduce I/O for joint queries.
1. **Table Partitioning（表分区）**
   Splitting a table into smaller parts stored separately, improving manageability and query performance.

数据字典与 Oracle 存储（Data Dictionary and Oracle Storage）

1. **Data Dictionary（数据字典）**
   A system catalog storing metadata about database objects, schema, and constraints.
1. **Oracle Logical Storage（Oracle 逻辑存储）**
   Oracle’s internal structure: blocks (basic I/O units), extents (contiguous blocks), segments (extent groups), and tablespaces (segment containers).
1. **Oracle Physical Storage（Oracle 物理存储）**
   The external representation of data on OS, independent of logical structure.

八、数据库系统基础（Database System Basics）

1. **Database Management System (DBMS, 数据库管理系统)**
   A software system managing interrelated data and providing tools for data access, including a database and access programs.
1. **Database（数据库）**
   A collection of structured data that is valuable, large, and accessed by multiple users simultaneously.
1. **File System Drawbacks（文件系统缺点）**
   1. **Data Redundancy（数据冗余）：** Duplicate information in multiple formats.
   1. **Access Difficulty（访问困难）：** Requires custom programs for new tasks.
   1. **Atomicity Issues（原子性问题）：** Partial updates cause inconsistencies.
   1. **Concurrency Anomalies（并发异常）：** Uncontrolled access leads to data corruption.

数据库结构与抽象（Database Structure and Abstraction）

1. **Data Abstraction Levels（数据抽象层级）**
   1. **Physical Level（物理层）：** How data is stored on disk.
   1. **Logical Level（逻辑层）：** Data structure and relationships.
   1. **View Level（视图层）：** Application-facing data with security restrictions.
1. **Schema vs. Instance（模式与实例）**
   1. **Schema（模式）：** The logical structure of the database, like a data type.
   1. **Instance（实例）：** Actual data in the database at a specific time.
1. **Data Independence（数据独立性）**
   1. **Physical Independence（物理独立性）：** Modifying storage without changing logical schema.

数据库组件（Database Components）

1. **Storage Manager（存储管理器）**
   Manages low-level data storage, providing interfaces for efficient retrieval and update.
1. **Query Processor（查询处理器）**
   Parses, optimizes, and executes queries, converting them to execution plans.
1. **Transaction Manager（事务管理器）**
   1. **Atomicity & Durability（原子性与持久性）：** Ensures transactions complete or abort fully.
   1. **Concurrency Control（并发控制）：** Prevents interference between concurrent transactions.

数据模型与语言（Data Models and Languages）

1. **Data Models（数据模型）**
   1. **Relational Model（关系模型）：** Organizes data in tables with rows and columns.
   1. **Entity-Relationship (ER) Model（实体 - 关系模型）：** Graphical design tool for database schema.
   1. **Object-based Model（基于对象模型）：** Combines OO principles with relational data.
1. **Database Languages（数据库语言）**
   1. **DDL (数据定义语言)：** Defines schema, constraints, and authorization (e.g., CREATE TABLE).
   1. **DML (数据操作语言)：** Queries and modifies data, including procedural and declarative types.

数据库设计（Database Design）

1. **Design Process（设计流程）**
    1. **Logical Design（逻辑设计）**：Choosing relation schemas and attributes.
    1. **Physical Design（物理设计）**：Deciding storage and index layouts.
1. **Normalization（规范化）**
    1. Eliminates data anomalies by splitting relations into higher normal forms (e.g., BCNF, 3NF).
1. **Client-Server Architecture（客户端 - 服务器架构）**
    1. **Two-tier（两层架构）**：Direct client-database connection.
    1. **Three-tier（三层架构）**：Adds an application server for middleware.

历史与应用（History and Applications）

1. **Database Evolution（数据库发展）**
    1. **1960s-1970s**：Relational model invented by Ted Codd.
    1. **1980s**：SQL becomes standard; parallel databases emerge.
    1. **2000s**：NoSQL and big data systems (e.g., Hadoop, BigTable).
1. **Real-world Applications（实际应用）**
    1. **Banking（银行业）**：Customer accounts and transactions.
    1. **Education（教育领域）**：Student registration and grades.
    1. **E-commerce（电子商务）**：Product catalogs and purchases.

其他概念（Other Concepts）

1. **Data Dictionary（数据字典）**
   Stores metadata about the database schema and statistics.
1. **Authorization（授权）**
   Controls user access to database objects via privilege management.
1. **Integrity Constraints（完整性约束）**
   Rules ensuring data correctness (e.g., referential integrity, domain constraints).
   以上概念覆盖了数据库系统的基础架构、设计原则及实际应用。若需对特定概念进一步解释或调整表述，可随时告知。